

INFORMÁTICA

[Área personal](#) / [Mis cursos](#) / [\(29185\) INFORMÁTICA](#) / [General](#) / [Examen ordinario](#)

Examen ordinario

Este curso la asignatura tiene dos modalidades de evaluación. El examen ordinario solo debe realizarse si optas por la Opción 1 de evaluación. En ese caso la nota del examen contribuirá en un 40% a la calificación global. Mira la [Presentación de la asignatura](#) en caso de dudas.

Envía la solución al examen como si se tratara de un ejercicio de laboratorio. No se exige que pase ninguna prueba, pero debe ser sintácticamente correcto para que sea admitido.

No utilices eñes ni tildes en los símbolos, *Skulpt* no los admite. Si tienes problemas para que sea aceptado encierra todo en una gran cadena de texto de la siguiente forma.

```
"""  
Tu programa aquí.  
"""
```

El examen consiste en la elaboración de 4 ejercicios de muy baja complejidad. Cada ejercicio puntúa 2.5 puntos.

Todos los ejercicios se evaluarán siguiendo la guía de calificación que se muestra al final de esta página.

1. Filtro de mediana (1D)

La mediana de un conjunto de números de cardinal impar es el valor que quedaría en la posición central si se ordenaran los números. Si el conjunto de valores es de cardinal par, entonces se suele hacer la media de los dos valores centrales.

Sea otra lista larga de números reales que llamaremos señal discreta. Se denomina filtro de mediana a una función que transforma todos los valores de la señal en la mediana de los valores vecinos. En términos matemáticos, si s_j es el valor de posición j en la señal, entonces el filtro de mediana sustituye cada valor s_j por $\text{mediana}(s_{j-K} \dots s_{j+K})$ donde K es un número relativamente pequeño que limita los $2K+1$ valores que se consideran vecinos. Al valor $2K+1$ se le llama tamaño de ventana.

Elabora la función `filtro_mediana(s, w)` que devuelve el resultado de aplicar el filtro de mediana a la señal s con tamaño de ventana w . Se asume que w es impar.

2. Conversión de imágenes de RGB a escala de grises

Sea una imagen definida como una matriz de puntos. Es decir, una lista de filas donde cada fila es una lista de puntos. En una imagen en color cada punto se representa por una tupla de tres valores enteros positivos (entre 0 y 255) que representan la intensidad de rojo, verde y azul respectivamente. En una imagen en escala de grises cada punto se representa por un valor entero positivo (entre 0 y 255) que define su intensidad (luminancia).

La luminancia de un punto en color se puede calcular con la fórmula siguiente:

$$Y = 0.299 R + 0.587 G + 0.114 B$$

Donde R , G y B son los valores para el punto de rojo, verde y azul respectivamente.

Elabora una función `rgb_to_grayscale(img)` que devuelve la representación en escala de grises de la imagen en color que se pasa como argumento.

3. Filtro binario (2D)

En procesamiento de imagen es frecuente la operación de umbralización, o filtrado binario. Consiste en sustituir todos los puntos fuera de un rango determinado por un valor 0 (negro), y todos los puntos dentro de un rango determinado por un 255 (blanco).

Elabora la función `filtro_binario(img, umbral)` que devuelve el resultado de umbralizar la imagen en escala de grises que se pasa como argumento, asignando 0 a todos los pixels que no llegan a umbral y 255 a todos los pixels que superan o igualan el umbral. La imagen se representa igual que en el ejercicio anterior.

4. Problema del viajante con restricciones

Sea un grafo modelado como una secuencia de arcos. Cada arco se representa con una tupla `(a, b)` donde `a` y `b` son los nodos conectados. El grafo es no dirigido, ambos sentidos de recorrido de los arcos están permitidos. Cada nodo se representa con un número entero.

Elabora la función `traverse(g)` que devuelve la secuencia de nodos del grafo `g` a visitar, de manera que se visiten todos los nodos y nunca pase dos veces por el mismo nodo. En caso de que no exista esta secuencia, debe elevar una excepción `ValueError`. Si en esta secuencia aparecen dos nodos consecutivos `a` y `b`, entonces debe existir en el grafo `g` el arco `(a,b)` o el `(b,a)`.

```
>>> G = ((1,2),(1,4),(4,3))
>>> traverse(G)
(2,1,4,3)
```

En el caso del ejemplo también es admisible el resultado `(3,4,1,2)`.

Estado de la entrega

Número del intento	Este es el intento 1.
Estado de la entrega	Enviado para calificar
Estado de la calificación	Sin calificar
Fecha de entrega	jueves, 17 de enero de 2019, 12:00:00
Tiempo restante	La tarea fue enviada 17 segundos antes

Criterio de calificación

Guía de evaluación para ejercicios largos, que supongan trabajo de programación real, con decisiones sobre organización, algoritmo, etc.

No se aplica a los retos, que por su singularidad se evalúan de forma diferente.

Corrección funcional El programa debe hacer exactamente lo que dice el enunciado. Puntuación máxima 75	Código no redundante La redundancia es una de las fuentes de errores más importantes en los programas de la vida real. Debe compartirse adecuadamente el código, creando funciones auxiliares si es necesario y usando apropiadamente las construcciones que aporta el lenguaje. Puntuación máxima 10	Código bien organizado Es esencial que el programa sea fácil de leer para otro experto. Si necesitas poner un comentario probablemente lo puedes hacer mejor usando funciones auxiliares, usando nombres adecuados, o descomponiendo en problemas más sencillos. Escribe funciones pequeñas, no uses bucles anidados a menos que sean triviales, usa nombres con sentido, y escribe tu programa en el orden lógico de lectura (de mayor a menor nivel de abstracción). Puntuación máxima 10	Código eficiente La eficiencia no es esencial y menos en un curso de introducción. Pero es muy importante no añadir código innecesario ni complicar lo sencillo. Debes tomarte tu tiempo en analizar si todo el código que mandas es útil y si no sería mejor hacerlo de otra forma. En este apartado se evaluará esencialmente la economía de código. Puntuación máxima 5
--	---	---	--

+ (0 palabras)

```
#py3 1 passed / 0 failed

# === === #
#ejercicio1
def filtro_mediana(s, w):
    s = sorted(s)
    subtrozo = extrae(s, w)

def extrae(s, w):
    l = []
    k = (w -1)/2
    for i, pos in enumerate(s):
        if pos <= k:
            l.append(s[i])
    return l

def mediana(l):
    if len(l)%2 == 0:
        return ((l[int((len(l)/2)-1)]+l[int(len(l)/2)])/2)
    return l[len(l)//2]

#ejercicio2
def rgb_to_grayscale(img):
    sol = []
    for fila in img:
        filas_def = []
        puntos = [0.299*punto[0] + 0.587*punto[1] + 0.114*punto[2] for punto in fila]
        filas_def.append(puntos)
        sol.append(filas_def)
    return sol

#ejercicio3
def filtro_binario(img, umbral):
    sol = []
    for fila in img:
        filas_def = []
        for p in fila:
            punto = [0 if i < umbral else 255 for i in p]
            filas_def.append(punto)
            sol.append(filas_def)
    return sol

#ejercicio4

# === === #
Ran 1 tests, passed: 1 failed: 0
```

...

Usted se ha identificado como PABLO MANUEL MARTÍN ISABEL (Salir)
(29185) INFORMÁTICA

Español - Internacional (es)

English (en)

Español - Internacional (es)

Descargar la app para dispositivos móviles